

元学习项目

1. 元学习

1.1. 什么是元学习

1.2. 元学习的训练过程

1.3. MAML

1.3.1. 元学习如何运用到推荐算法的冷启动问题？

1.3.2. MAML训练过程（MAML是元学习中的一种特殊的训练框架）

1.4. Meta-Embedding（业内比较厉害的经典的基于元学习解决冷启动问题的模型，在快手抖音已经落地）

1.4.1. 预训练

1.4.2. id embedding生成器

1.4.3. 训练与预测

2. 我的项目

2.1. 项目概述

2.2. 优化点1：生成器构建

2.2.1. 如何为冷启动item找到相似的K个老item？

2.2.2. 生成id embedding

2.3. 优化点2：基于对比学习的特征空间优化

2.3.1. 子集划分方式

2.4. 优化点3：嵌入去噪

2.5. 优化/训练流程

2.6. 预测流程

3. 实验效果

3.1. 数据集（建议自己再找三个）

3.2. 消融实验（下图是我的论文，这里你要把你论文的三个模型分别做消融实验，实验效果直接编就行，自...

3.3. 效果提升了多少？

4. 训练配置说明

5. 数据说明

5.1. 第一种类型的数据

5.2. 第二种类型的数据

6. 代码说明

7. 简历怎么写

8. 常见面试问题

补充：

1. 本文提到的嵌入就是embedding。

点击率预估是推荐系统最核心的任务之一，近些年来，深度学习的出现极大地提高了点击率预估任务的准确度。（你要是学过深度学习应该知道embedding这个东西，数据越多，embedding训的越好）。但是对于冷启动item（新item）来说，他的数据不足，那么就会导致他的embedding训练的很差，进而导致推荐模型推不准。

推荐模型范式：



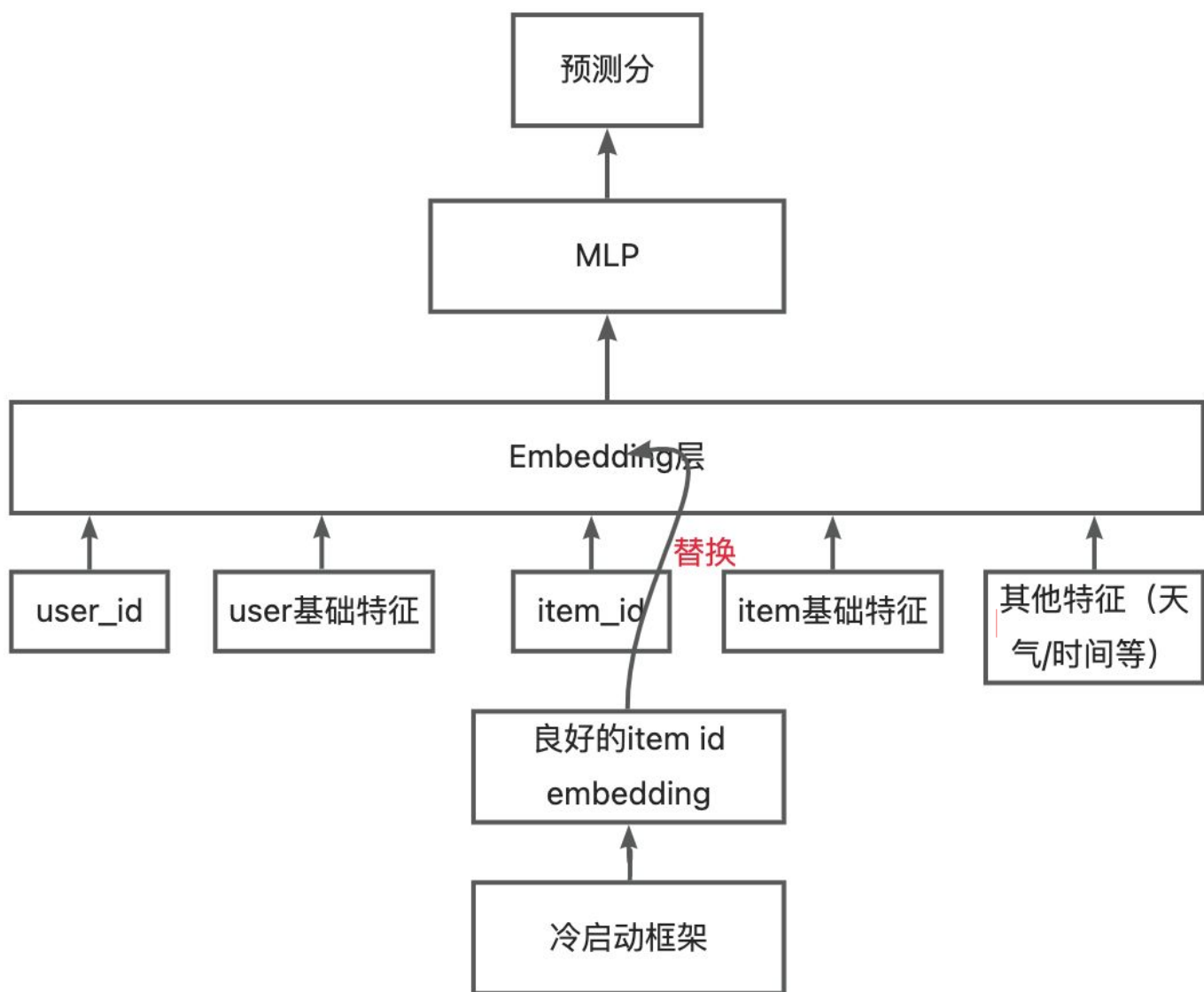
对图的解释：最底层是输入的特征，把这些特征都搞成embedding之后，输入MLP(这个就太基础了，如果不知道的话可以上网查查)，最后得到输出结果，即一个预测分，即预测当前user点击当前item的概率。既然有预测值，就会有真实值，1代表点击，0代表未点击。有预测值和真实值以后，代入交叉熵损失函数，算loss，然后梯度下降，反向传播，更新MLP和Embedding的参数。所以数据越多，这些参数被更新的次数就越多，模型的效果就越好。

那为什么这么多特征，只有ID embedding训不好呢？因为id特征是每个item独有的，是一个数字，耐克的篮球是1，李宁的篮球是2，每个item都不会重复。所以id特征只能吃到自己的数据，在自己数据不足的情况下，就更新不好。但其他特征，比如类别=运动，这个特征可以吃到所有运动的item的数据，所以肯定更新的很好。

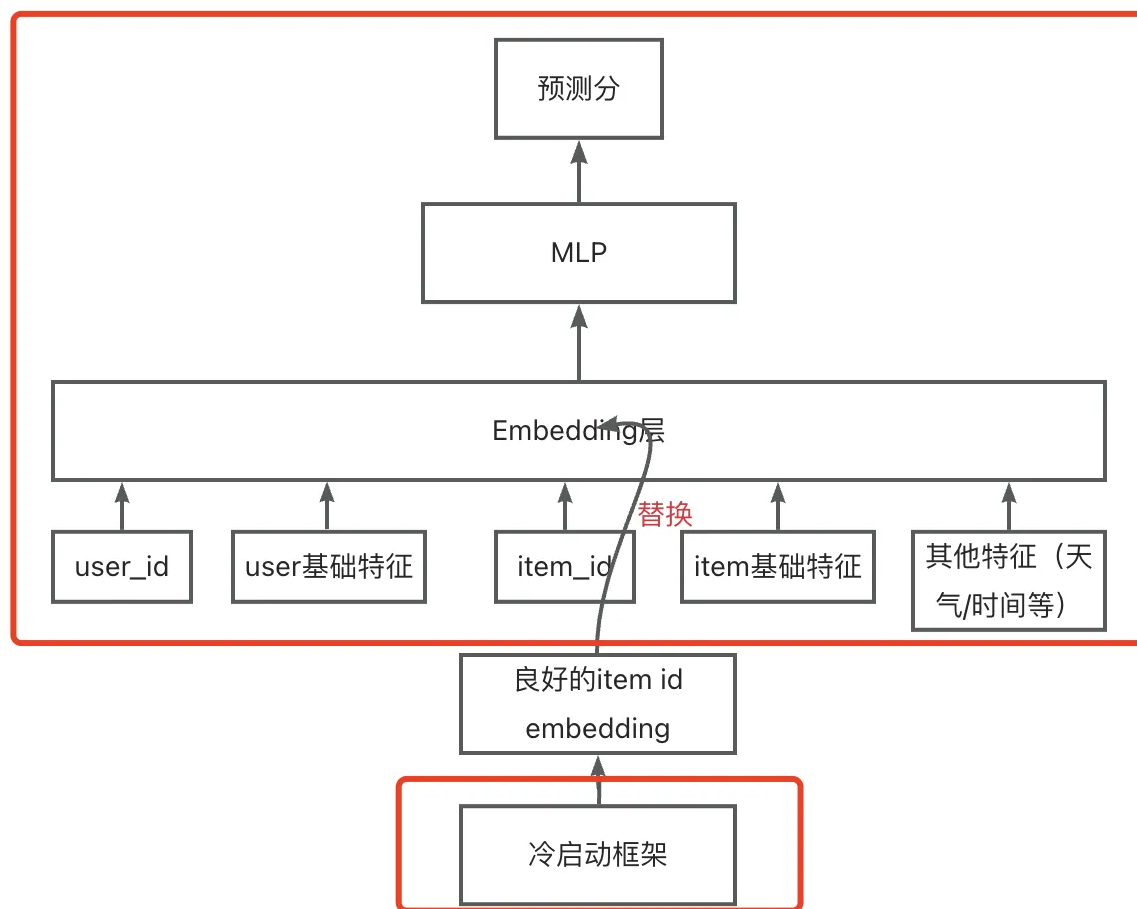
所以，解决冷启动问题的本质，就是解决冷启动item ID embedding训练不好的问题。（当然user也有冷启动问题，用这套方法也能解决）

所以，我们要研究一个冷启动框架，帮助新item生成良好的id embedding，来替换它之前垃圾的id embedding。如下图：我们设计了一个冷启动框架，生成一个良好的初始id embedding，

来替换之前随机初始化的id embedding，以此来提高推荐效果。



为了表达方便，暂且将参数分为两部分称呼：**推荐模型的参数**、**冷启动框架的参数**。这样划分：



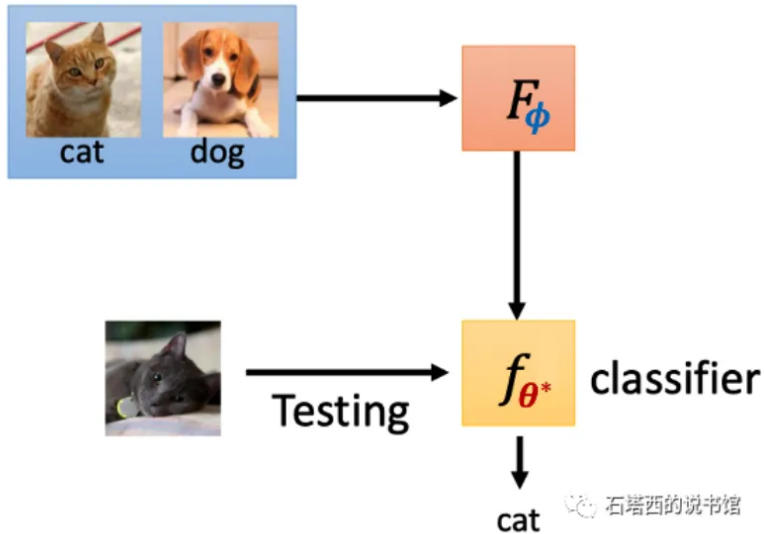
推荐模型的参数怎么训练，上面已经讲了，那么冷启动框架的参数怎么训练呢？用元学习。请往下看

1. 元学习

1.1. 什么是元学习

先回顾一下，常规的机器学习是怎么做的？

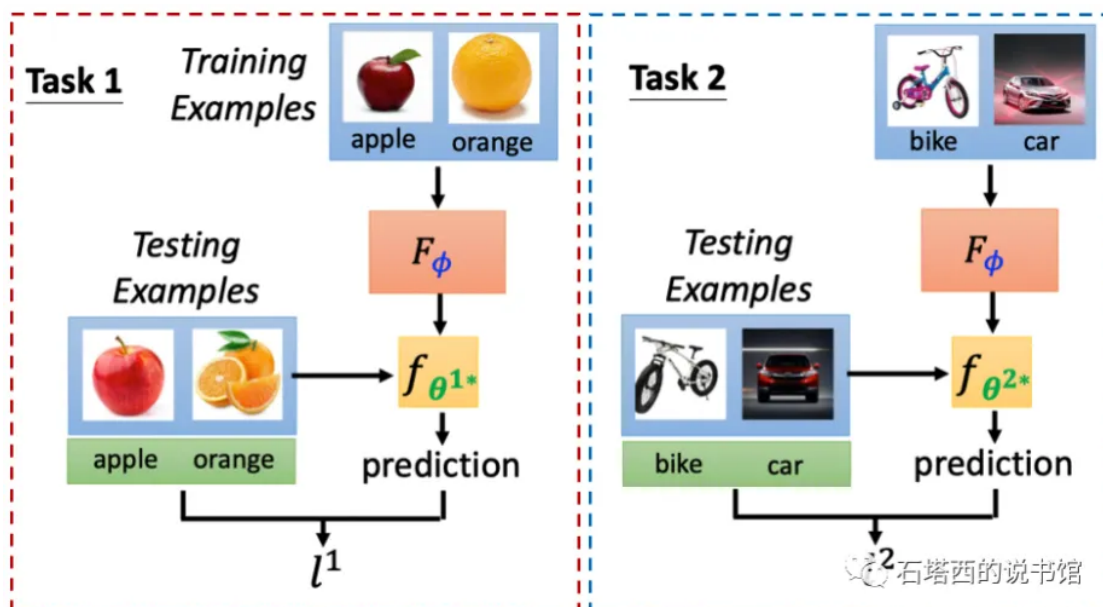
Training Examples



F_ϕ 是一个函数模板，暂时没有任何功能。给它喂入一条条样本进行训练，它的参数得到更新，就可以得到具体的模型 f_{θ^*} ， θ^* 是训练出来的最优参数。你喂的是猫的图片， f_{θ^*} 就是猫分类器，喂的是狗的图片， f_{θ^*} 就是狗分类器。

1.2. 元学习的训练过程

- 对Meta Learning来说，训练单位不再是一条条单独的样本，而是一个个"任务" (Task) 。一个Task是由多条数据组成的，这多条数据又可以划分成两个数据集，**支撑集**（元学习的训练集）和**查询集**（元学习的测试集）。
- 第一个Task是用于分类水果图片。训练Task 1时，将其中的Task 1支撑集（一批水果图片和标注）喂入模板 F_ϕ ，训练得到一个水果分类器 $f_{\theta_1^*}$ ， θ_1^* 是训练得到的最优权重。这个步骤只用到单独一个Task的数据。



- 再将Task 1中的查询集喂入训练好的模型 $f_{\theta_1^*}$ ，计算出在查询集上的损失 l_1 。
- 同理，将Task 2中的支撑集（交通工具的图片和标注）喂入模板 F_ϕ ，训练得到一个交通工具分类器 $f_{\theta_2^*}$ ， θ_2^* 是训练得到的最优权重。再拿Task 2中的查询集喂入 $f_{\theta_2^*}$ ，计算得到查询集的损失 l_2
- 假设训练一个批次（Batch）有N个任务，那么这个batch的总loss就是所有任务的查询集上的损失之和，上面的例子中，总loss= $l_1 + l_2$ 。具体公式如下：

$$L_{meta}(\phi) = \sum_{n=1}^N l_n = \sum_{n=1}^N L(D_n^{test} | \theta_n^*)$$

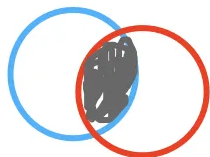
- L是所有任务共用的损失函数
- l_n 是第n个任务在其测试集上的损失
- D_n^{test} 是第n个任务中的测试集（即Query Set）
- θ_n^* 是第n个任务训练得到的最优参数

有了损失函数 $L_{meta}(\phi)$ ，如何用这个损失去更新参数呢？用下面讲的MAML。

1.3. MAML

1.3.1. 元学习如何运用到推荐算法的冷启动问题？

- 通过若干组任务的学习（比如Task 1是分辨不同水果，Task 2是分辨不同的交通工具），为模型学习出一套高质量的参数初值。（如下图，本质上是学到了各个任务之间的共性，这个共性就是“高质量的初始参数”）



当面对一个新任务（比如分辨不同动物时）时，由这套高质量的参数初值出发，只需要经过少量样本的迭代，就能达到适合新任务的最优参数 θ^* ，从而解决了新任务样本不足的问题。

- 如何套到我们的冷启动问题里呢？我们把学习一个item的id embedding当做一个任务，我们用多个老item的数据构造多个task：task1是学习篮球的id embedding，task2是学习足球的id embedding.....，通过对不同task的学习，冷启动框架已经学习到了“生成item id embedding”的能力，当新item排球来的时候，我们直接用冷启动框架为它生成一个id embedding，这个id embedding质量就还不错了。（在工业界，如果没有冷启动框架，新item来的时候，它的id embedding是随机初始化的）

1.3.2. MAML训练过程（MAML是元学习中的一种特殊的训练框架）

Algorithm MAML

Input: step-size α // 由初值 ϕ 经过一步迭代得到最优参数 θ^* 的步长

Input: step-size β // 由 ϕ 的老值迭代成新值的步长

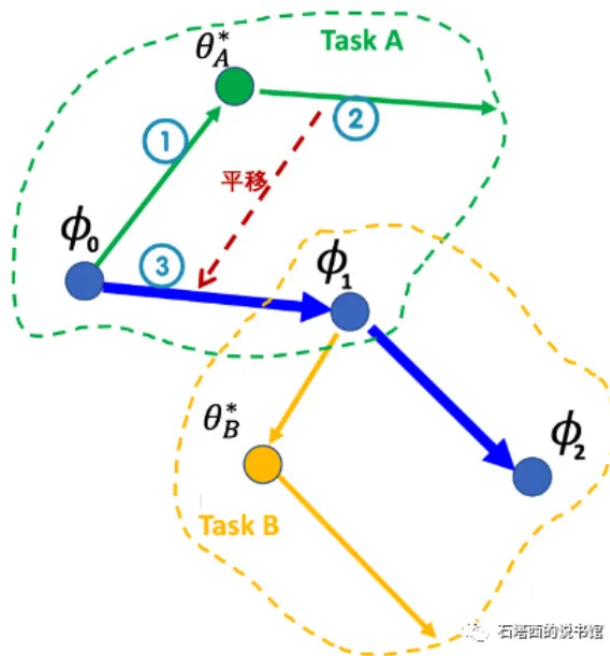
```
1. randomly initialize  $\phi$ 
2. while not done do
3.   given a batch of Tasks,  $T_n, n \in \{1, \dots, N\}$ 
4.   for each  $T_n$  do // Within-Task Learning
5.     //在 Task  $n$  的训练集 $D_n^{\text{train}}$ 上, 由初值 $\phi$ 经过一次梯度下降, 得到 Task  $n$  的最优参数 $\theta_n^*$ 
6.      $\theta_n^* = \phi - \alpha \frac{\partial L(D_n^{\text{train}}|\phi)}{\partial \phi}$  //(8-17)公式(a)
7.     //在 Task  $n$  的测试集 $D_n^{\text{test}}$ 上, 在最优参数 $\theta_n^*$ 处计算梯度, 为元学习做准备
8.      $g_n^{\text{meta}} = \frac{\partial L(D_n^{\text{test}}|\theta_n^*)}{\partial \theta_n^*}$  //第  $n$  个任务对元学习贡献的梯度
9.   end for
10.  // Across-Task Learning
11.   $\phi = \phi - \beta \sum_{n=1}^N g_n^{\text{meta}}$  //元学习更新迭代初值, 公式(8-17)(b)和(c)
12. end while
```

石塔西的说书馆

注意, 是计算完一个batch内的所有task, 将它们的梯度加起来, 用于更新冷启动框架的参数 ϕ 。

假设一个batch只有一个task的话, 训练过程如下:

- ϕ_0 是初值 ϕ 的初值, 第一个批次是训练Task A。
- 第一步: 由初值根据Task A中的训练集梯度下降一步, 得到Task A的最优参数 θ_A^*
- 第二步: 根据Task A的测试集, 计算损失对最优参数 θ_A^* 的梯度。
- 第三步: 将第2步计算出来的损失对最优参数 θ_A^* 梯度, 平移过来成为损失对初值的梯度, 沿此梯度方向迭代一步, 得到初始值的新值 ϕ_1 。
- 然后开始第二个批次, 训练Task B, 从 ϕ_1 出发, 也是同样的道理, 再pua一次。经过N次pua, 就得到了一个良好的初始化参数。



测试集：

为什么要这样更新参数呢？人话版：模型是很naive的，它无法预知样本上的多个变化。在任务样本比较少的情况下，模型把这几个样本（支撑集）拟合好了以后尾巴就翘上天了，“嗨，轻轻松松就拟合了”。这时候一次性抛出一堆query set（查询集），瞬间就傻眼了，原来我做的还不够好。

（正常深度学习的训练方式，模型无法感知，因为训练数据少，自己对某个Task已经过拟合了。所以通过先用支撑集训，再给它抛查询集的方式，去让模型感知，它过拟合了。这里补充一句，如果数据少又不做处理的话，推荐效果不好，这叫冷启动问题，但本质原因是因为数据太少模型太复杂，导致了过拟合，所以推荐效果不好）

给学生讲解时的笔记：

ϕ_0 到 θ_A^* ，如果对于一个正常的item来说，是没有问题的。但是我们研究的是冷启动问题，它没有足够多的数据，去让 θ_A^* 训练的很牛逼。所以，用很少的数据训练出来的 θ_A^* 一定是过拟合的。过程：给模型一点点数据（训练集），模型一看，这么简单，直接更新跑到了 θ_A^* 的位置，然后我们让它涨涨教训，给他一个测试集测一下，傻眼了，loss很高，模型会反思，奥，我刚刚跑到过拟合的地方去了，没把参数更新好。所以，把这个惩罚（测试集算出来的loss）更新到刚刚开始

的模板参数 ϕ_0 ，pua模型，以后别给我跑到过拟合的地方去。经过很多任务的学习，模型再也不敢跑到过拟合的地方去了。

补充（本文用的MAML和经典的MAML有出入，经典的MAML只会用到第二阶段的loss，本文第一阶段第二阶段的loss都会用，所以看懂本文的训练过程以后，和经典的MAML对不上是正常的）

1.4. Meta-Embedding（业内比较厉害的经典的基于元学习解决冷启动问题的模型，在快手抖音已经落地）

1.4.1. 预训练

在开始元学习之前，先用老item的样本预训练出一个CTR模型（例如DNN）。

这里是推荐系统的基础知识：DNN=Embedding + MLP

对于老item来说，数据充足，其id embedding和其他特征embedding都被训练的很好，推荐效果是ok的。但对于冷启动item来说，其他特征embedding的可以复用老item的（例如类别中的“运动”这个特征的embedding，新的运动item和老的运动item都共用一个运动embedding），但id embedding是自己特有的，如果item本身的数据量不足，导致id embedding得不到很好的训练，进而导致推荐效果差。

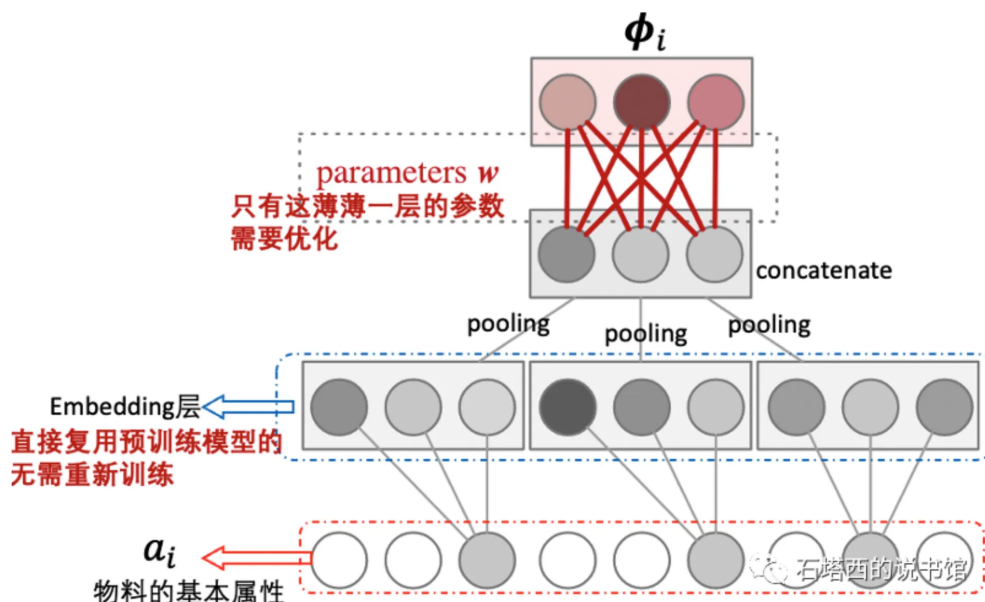
为了解决这个问题，用元学习的方式训练一个id embedding生成器，等冷启动item来临时，可以用生成器为它生成一个良好的id embedding。那这个id embedding生成器怎么训练呢？用多个老item的数据构造多个task，用MAML的方式训练一个embedding生成器，用于给新item生成id embedding。

1.4.2. id embedding生成器

输入：item的常规特征（品牌，类目）

输出：item的比较不错的id embedding的初始值

（因为冷启动item的id embedding不是不好吗？那我构造一个生成器，给你生成一个好的。）



1.4.3. 训练与预测

训练

深度推荐模型（DNN）和元学习框架是分开训练的。

- 第一步，先把DNN训练好，得到不错的参数（包括网络参数+embedding矩阵，但不包括生成器的参数）。
- 第二步，然后把第一步得到的参数冻结，用老item的数据去训练生成器的参数。

（如果这里看不懂的话不要紧，我们的方案和他用了相同的方式，下面有详细介绍）

预测

预测集都是冷启动的item（因为要老item的id embedding已经训的很好了，根本不需要你为他们生成id embedding）。首先用生成器为冷启动item生成一个不错的初始化id embedding，用不错的id emb替换冷启动item原来的垃圾的id emb，带到DNN里去预测。

2. 我的项目

2.1. 项目概述

项目名称：基于元学习个性化的冷启动嵌入框架（合理即可，没啥对不对的，你也可以自己改）

优化点有四个：

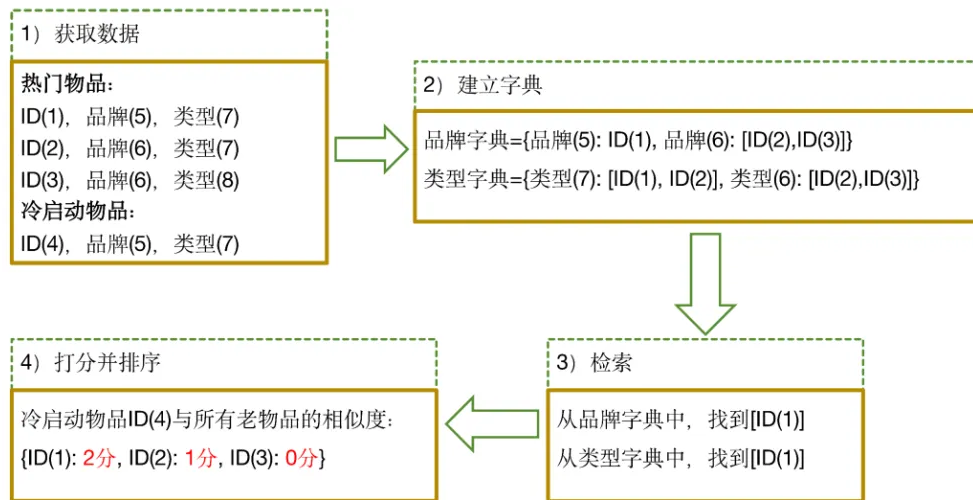
1. 构建了生成器，除了基础特征外，额外引入了老item的信息，并用MAML的方式去训练它。
2. 用对比学习优化了特征空间。
3. 用点击序列做了去噪。
4. 用了比较流行的MAML来训练了冷启动框架

2.2. 优化点1：生成器构建

我设计了 id 嵌入生成器为冷启动item生成良好的初始化id嵌入，生成器的输入是item的属性特征和群组特征，输出是良好的初始 id 嵌入。回答：首先，给定一个冷启动item，找到和它在属性级别上最相似的 K 个老item，老item的 id 嵌入是用大量的数据训练出来的，是值得信赖的。然后用相似 K 个老item的平均嵌入来表征冷启动item属于的item群组，得到群组特征嵌入。群组嵌入和冷启动item的属性特征嵌入一起输入到id embedding生成器中，得到冷启动item的良好的初始id嵌入。

2.2.1. 如何为冷启动item找到相似的K个老item？

给定一个冷启动item，通过它的属性特征找到和它最相似的 K 个老item。判断相似的方法是，如果有一个老item和当前冷启动item有 2 个属性特征相同，那么这个老item的相似度得分就是 2。给所有老item打分后，然后取得分最高的 K 个老item，就得到了和当前冷启动item最相似的 K 个老item。具体检索方法，下图中给出一个例子：



- 获取数据：id 为 1、2、3 的是热门物品，id 为 4 的是冷启动物品。物品的属性特征有两个：品牌和类型。
- 为所有属性特征建立字典。key 是属性值，value 是等于这个属性值的物品 id 集合。本例中的品牌特征字典：品牌值为 5 的有物品 1，品牌值为 6 的有物品 2和物品 3。类型特征字典同理。
- 为冷启动物品检索相似热门物品。本例中，冷启动物品的品牌值为 5，和它品牌值相等的热门物品有物品 1；冷启动物品的类型值为 7，和它类型值相等的热门物品有物品 1 和物品 2。
- 给热门物品打分并按照分数排序。本例中，热门物品 1 有两个属性特征和冷启动物品相等，所以相似度得分为 2，热门物品 2 的得分为 1，热门物品 3 的得分为 0。如果 k=2，则取物品 1 和物品 2。

找到相似的老item集合后，把他们的id embedding平均池化：

$$\mathbf{f}_{v_i}^0 = \text{mean-pooling}(V)$$

$\mathbf{f}_{v_i}^0$ 代表群组特征的embedding，V代表老item们的embedding的集合，mean-pooling代表平均池化。

2.2.2. 生成id embedding

把群组特征和item其他的属性特征连起来：

$$\mathbf{z}_i = \mathbf{f}_{v_i}^0 || \mathbf{f}_{v_i}^1 || \cdots || \mathbf{f}_{v_i}^n$$

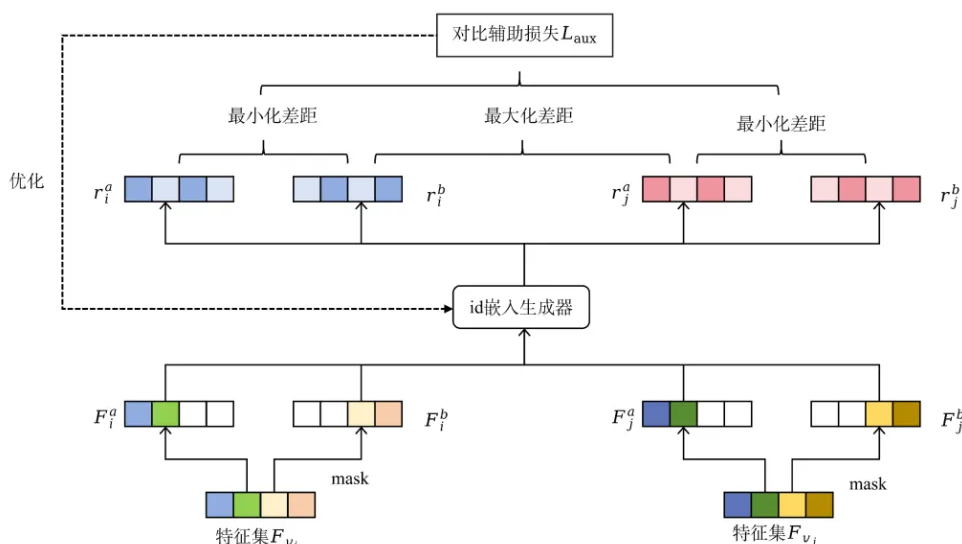
(item自己本身的属性特征的emb和群组特征的embedding concat一起)

再输入到生成器中（本质就是一个MLP）：

$$\mathbf{r}_i = \gamma \tanh(\mathbf{W} \mathbf{z}_i)$$

r_i 就代表生成器生成的良好的初始id embedding。忽略 γ ，没啥用。

2.3. 优化点2：基于对比学习的特征空间优化



首先，对于每个冷启动物品 v_i ，把它的属性特征集合 $F_{v_i} = \{f_{v_i}^1, f_{v_i}^2, \dots, f_{v_i}^n\}$ 分为两个子集： F_i^a 和 F_i^b 。然后把 F_i^a 输入到 3.3.1 节构造的 id 嵌入生成器中，缺失的部分用默认值 0 填充，得到 id 嵌入 r_i^a ，同理，把 F_i^b 输入 id 嵌入生成器得到 id 嵌入 r_i^b ，具体如公式 3-8，公式 3-9 所示：

$$r_i^a = \text{generator}(F_i^a) \quad (3-8)$$

$$r_i^b = \text{generator}(F_i^b) \quad (3-9)$$

所以，对于每个冷启动物品 v_i 来说，可以得到两个 id 嵌入 r_i^a 和 r_i^b 。在属性特征不全的情况下，期望模型可以识别 r_i^a 和 r_i^b 代表了同一个物品。具体的优化目标有两个：对于相同的冷启动物品 v_i ，期望最小化 r_i^a 和 r_i^b 的差距；对于不同的冷启动物品 v_i 和 v_j ，期望最大化它们的 id 嵌入 r_i^b 和 r_j^a 的差距。

定义 (r_i^a, r_i^b) 为一个正样本对, (r_i^b, r_j^a) 为一个负样本对, 且 $i \neq j$ 。使用余弦相似度衡量两个 id 嵌入向量 r_i^b 和 r_j^a 的相似度:

$$s(r_i^b, r_j^a) = \frac{r_i^b \cdot r_j^a}{\|r_i^b\| \cdot \|r_j^a\|} \quad (3-10)$$

其中, 分子是两个 id 嵌入向量的点积, 分母是两个 id 嵌入向量的模长的乘积。为了完成两个优化目标, 设计了辅助损失 L_{aux} 如下:

$$L_{aux} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp\left(\frac{s(r_i^a, r_i^b)}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{s(r_i^b, r_j^a)}{\tau}\right)} \quad (3-11)$$

损失函数解释: 分子是item自己产生的两个embedding的相似度, 它们互为正样本, 分母是item自己的一个embedding和其他item产生的embedding的相似程度, 其他item的embedding可以看作是当前item的embedding的负样本, N是负采样的个数。根据对比学习的思想, 分母是越小越好, 分子越大越好, 整体就是越大越好, 但是Loss必须是越小越好, 所以前面还加了负数。

形式上, 辅助损失 L_{aux} 是 softmax 形式, 负样本的距离越近, 给它的惩罚越大, 这样可以使模型更加注重那些困难负样本(和正样本距离很近, 相似度高的负样本)。其中, τ 是温度系数, 用于控制惩罚的力度, τ 越小, 困难负样本和简单负样本经过 softmax 输出后的差异也越大, 对困难负样本的惩罚力度也就越大。“同源相近, 异源相远”可以使特征空间分布得更均匀, 有利于提升建模效果, 所以理论上, 温度系数 τ 越小越好。但实际实验中发现, τ 逐渐变小的过程, 实验效果先上升后下降。分析原因是存在一些特殊情况, 困难负样本其实是潜在的正样本。例如, 毛衣和羊毛衫很相似, 它们的 id 嵌入本就是相似的。但羊毛衫对于毛衣来说是困难负样本, 如果惩罚力度过大, 使毛衣和羊毛衫的 id 嵌入过于不相似, 违背了事实, 从而影响了建模效果。所以, 温度系数 τ 的取值, 要由灵敏度分析实验结果决定。

面试题: 如何衡量相似度?

面试题: 讲一下辅助损失怎么设计的? (必问)

2.3.1. 子集划分方式

冷启动物品的属性特征集要先划分为两个子集, 才能执行上述的数据增强任务。不同的划分方式, 会导致不同的结果。不同的特征之间存在相关性, 如果划分后的两个子集很多特征都高度相关, 那

它们通过生成器生成的 id 嵌入也很容易相似，这就失去了对比学习的意义。为了解决这个问题，首先在冷启动物品的特征集中随机选择一个种子特征 f_{seed} ，然后利用余弦相似度计算和种子特征最相关的一半特征，然后把种子特征和这一半特征作为一个子集，其余特征作为另一个子集。这样做可以保证两个子集的信息差异最大化，更有利于基于对比学习的数据增强任务。

面试题问题：具体怎么划分的呀？

首先在冷启动物品的特征集中随机选择一个种子特征 f_{seed} ，然后利用余弦相似度计算和种子特征最相关的一半特征，然后把种子特征和这一半特征作为一个子集，其余特征作为另一个子集。这样做可以保证两个子集的信息差异最大化，更有利于基于对比学习的数据增强任务。

2.4. 优化点3：嵌入去噪

推荐系统每天会产生大量的噪声，例如，用户的错误点击。对于冷启动item来说，交互用户的数量本来就是有限的，噪声用户对冷启动item的embedding学习的影响会非常大。本文提出用冷启动item的所有交互用户的平均embedding来增强冷启动item的 id embedding。

降噪因子 = $mean - pooling(\text{交互用户1的id emb}, \dots, \text{交互用户n的id emb})$

$$r_{en} = \text{生成器生成的id emb} + \text{降噪因子} = r_i + \text{降噪因子}$$

其实思想很简单，就是把当前冷启动item过去所有交互过的user的id embedding加起来，然后再除以n，取个平均而已。

为什么这么做？：平均池化可以减轻离群点对于效果的影响，从而起到降噪的作用。

最终，我们得到了最终的id embedding。

2.5. 优化/训练流程

备注：注意注意注意！下文提到的“生成器”指的是优化点1+优化点2+优化点3合起来的大框架，并不是只有优化点1的生成器！生成器所生成的id embedding指的是上述经过三个优化点后的

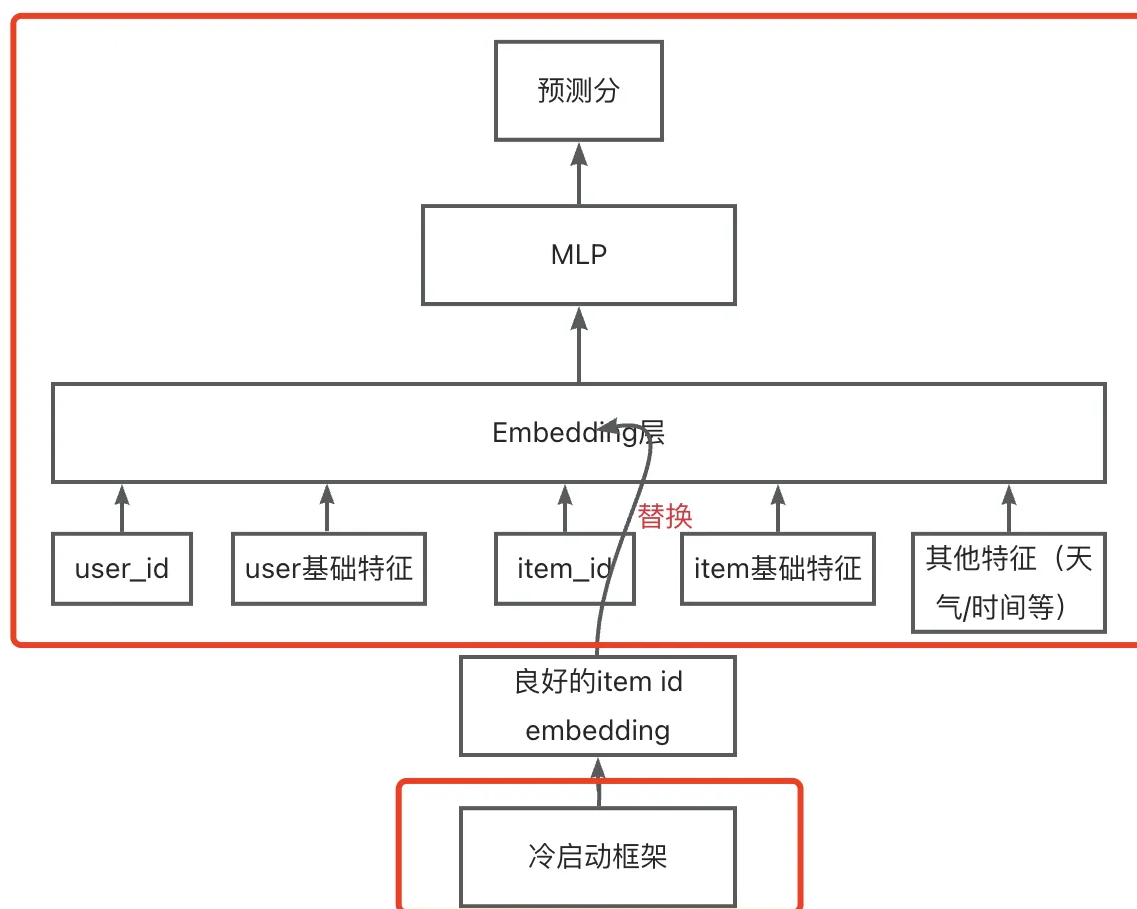
r_{en} 。

训练分为两阶段：

第一阶段：训练主网络的参数（随便选取一个基于MLP的推荐模型即可）。这一阶段的训练就很正常的推荐模型训练一样（预测值和真实值算个loss，然后梯度下降反向传播去更新参数）

第二阶段：冻结第一阶段的训练好的主网络参数（在第二阶段，第一阶段训好的参数就不更新了），再用MAML的方式训练冷启动框架的参数。（优化点1优化点2和优化点3）

即第一阶段训练上面红框的参数，第二阶段训练下面红框的参数。



数据划分：

首先，数据量大于512条的item，视为老item，小于等于512条的视为新item。（这里为什么是512呢？按照91分，90%的老item，10%的新item，卡阈值的时候刚好卡到512了）

这里的新、老item的划分其实比较粗糙，新item包括了冷启动item+长尾item，无奈之举，这是和数据集有关。因为电影数据集的话，是没有时间这个特征的，如果用的是阿里广告数据集，是

有时间这个特征的，到时候新item的划分标准就是“小于512条且最近3天产生”的广告。

所有老item的数据按照9:1的比例划分为两部分，记为Old1和Old2，注意这里的Old1和Old2的每条数据均是完整的数据，带着user特征、item特征和label的，并不是只有item特征。Old1用于第一阶段的训练，Old2用于第二阶段的训练（两部分数据交集空，因为推荐算法，所有数据只能训一遍，训第二遍的话会发生严重的过拟合）。所有新item的数据，作为最终的测试集（因为这个框架就是为了解决冷启动问题的，所以要评估冷启动的item是否推的更准了）。

第一阶段的训练没啥讲的，细致地讲讲第二阶段的训练。我们把冷启动过程分为两个阶段，在完全冷启动阶段，物品的交互数据为零。在预热（warm up）阶段，物品渐渐有了少量交互数据（以电影为例，完全冷启就是0个人看过，预热阶段就是10个人看过，但是还是很少，它的id embedding还是训不好。如果没有我们这个项目的話，通常做法是给item随机初始化一个id embedding先将就用着，但是我们用训练好的id embedding生成器给它生成一个还不错的初始id embedding，可以提高冷启动item的推荐效果）。所以我们的优化目标也就是这两个，在冷启动和预热阶段都要有一个好的预测效果，即

- 在完全冷启动阶段，期望给冷启动物品一个不错的初始id embedding，最小化冷启动的预测损失（0数据的时候）。
- 在预热阶段，冷启动物品有了少量的交互数据，期望最小化预热阶段的预测损失。

所以，最终我们的loss应该是两部分组成的，冷启loss+预热loss。上面说了，我们要用Old2去训练冷启框架，具体来说，按照元学习的思想：“以任务为单位来训练冷启框架”。所以我们把学习每个item的id embedding作为一个任务，这样Old2就能被划分为N个任务：为N个item分别学习他们的id embedding。在训练之前，我们先整理一波数据，每个item取两个batch的数据，每个batch大小为256，那么每个item要取512条数据。

训练流程：以第一个任务来说，我们先用冷启框架为item1生成他的id embedding，记为r，用它替换深度推荐模型中他原有的垃圾的id embedding（一般是随机初始化的），然后用推荐模型在item1的第一个batch的数据上得到预测结果（这一步是在模拟完全冷启动），进而能得到item1的cold start loss。然后让r在cold start loss上梯度下降一步，得到r'（这一步是在模拟，数据增多以后，id embedding被更新的过程），再用r'替换item1原有的id embedding，然后用推荐

模型在item1的第二个batch上得到预测结果，进而得到warm up loss。最后，总loss=cold start loss + warm up loss。然后用这个总loss去梯度下降更新冷启动框架的参数。

经过很多很多任务的训练，冷启动框架最后就被训练好了。当一个新item诞生，就用冷启动框架给他生成一个优质的初始的id embedding，进而可以提高推荐效果。

补充说明：为什么要用老item的数据即Old2去训练冷启动框架呢？

1.因为真正的冷启动item都被拿去当测试集了。

2.仔细看看Lcold和Lwarm这俩loss是怎么产生的，如果一条数据都没有，就不可能产生这俩Loss。所以对于冷启动item来说，产生这俩Loss很困难。所以，我们本质上是在用数据充足的老item去模拟了“冷启动item的一生”，即从完全冷启动到数据不断增多的一个状态。

又该上我的论文的图了：

本文使用热门物品去训练PMEC，让它为冷启动物品服务。所以，使用热门物品模拟冷启动物品数据不足的情况。具体方法为，对于每个热门物品 v_j ，从它的交互数据中取出不重复的两个batch的数据 D_a 和 D_b ，每个batch的数据量为 M 。首先，用PMEC为热门物品 v_j 生成初始id嵌入 r_{en} ，用它替换深度推荐模型中该热门物品训练不好的id嵌入，带入深度推荐模型在第一个batch的数据 D_a 进行预测，得到完全冷启动阶段的损失，具体如公式3-19所示：

$$L_{cold} = \frac{1}{M} \sum_{j=1}^M [-y_{aj} \log \hat{y}_{aj} - (1 - y_{aj}) \log(1 - \hat{y}_{aj})] \quad (3-19)$$

其中 $\hat{y}_{aj}$ 代表预测结果， y_{aj} 代表真实值， L_{cold} 是冷启动阶段的损失。为了让 r_{en} 在完全冷启动阶段和预热阶段都能有好的效果，让 r_{en} 在 L_{cold} 梯度下降的方向上再更新一步，具体如公式3-20所示：

$$r'_{en} = r_{en} - \eta \frac{\partial L_{cold}}{\partial r_{en}} \quad (3-20)$$

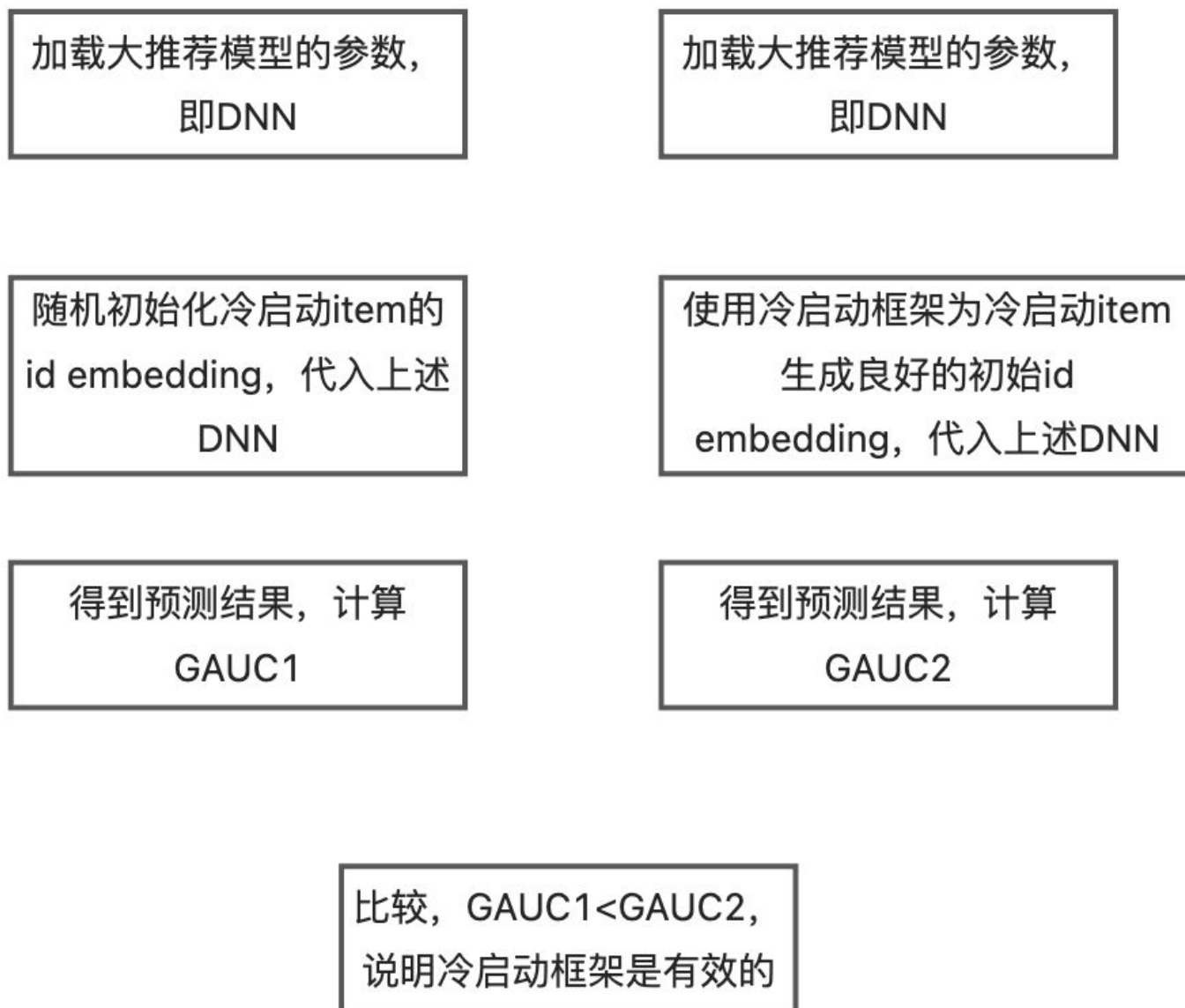
其中 η 是梯度下降的步长。然后模拟预热过程，用 r'_{en} 在第二个batch的数据 D_b 上进行预测，得到预热损失，具体如公式3-21所示：

$$L_{warm} = \frac{1}{M} \sum_{j=1}^M [-y_{bj} \log \hat{y}_{bj} - (1 - y_{bj}) \log(1 - \hat{y}_{bj})] \quad (3-21)$$

最后，把完全冷启损失、预热损失、嵌入空间优化的辅助损失结合起来，得到最终损失：

$$L = w_1 L_{cold} + w_2 L_{warm} + w_3 L_{aux} \quad (3-22)$$

2.6. 预测流程



3. 实验效果

3.1. 数据集（建议自己再找三个）

接着，介绍三个数据集的具体情况以及一些数据的预处理。

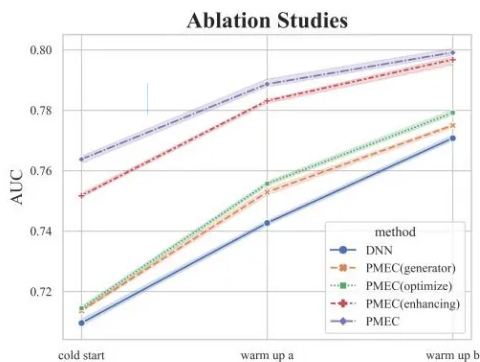
1) **MovieLens-1M 数据集**：该数据集是推荐系统经典的电影评分数据集，包含了 6000 名用户对上千部电影的评价。每条样本包含电影 id，用户 id 以及用户对电影的评分。电影有四个特征：电影 id，电影标题，电影类型和发布年份。用户有四个特征：用户 id，职业，性别和年龄。评分有 1-5 分，因为本文研究的点击率预估任务是二分类任务，所以把 3 分以上的评分转化为 1，视为正样本，3 分及其以下的评分转化为 0，视为负样本。

2) **Taobao 数据集**：该数据集是阿里巴巴的旗下的淘宝的广告展示的点击数据集。包含 2600 万条记录。每条样本包含广告 id，用户 id 及其标签。广告有五个特征：广告 id，品牌，广告计划 id，广告类别，广告主 id。用户有九个特征：用户 id，性别，年龄，微群 id，管理 id，职业，城市等级，消费能力，购物等级。标签有两个值，1 代表用户点击了广告，0 代表没有点击。

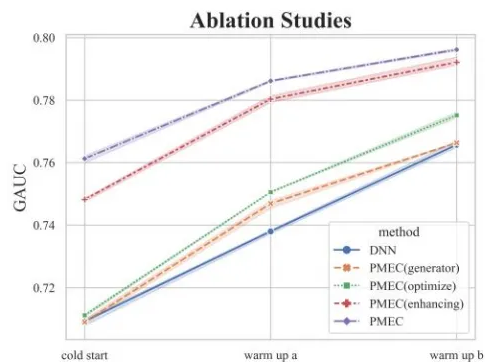
3) **CIKM 电商数据集**：该数据集来自阿里巴巴举办的 CIKM 2019 EComm AI 的比赛，是一个电商数据集。每个样本包含商品 id，用户 id 和行为标签。商品有四个特征：商品 id，商店 id，类别，品牌。用户包含四个特征：用户 id，购买力等级，性别，年龄。行为标签有四种取值：浏览，购买，加入购物车，收藏。购买是强正信号，其余行为都是弱正信号或负信号，所以把标签是“购买”的转化为 1，当做正样本，其余样本的标签全转化为 0，当做负样本。

3.2. 消融实验（下图是我的论文，这里你要把你论文的三个模型分别做消融实验，实验效果直接编就行，自圆其说即可）

PMEC 主要包含四个模块：生成器构建，嵌入空间优化，嵌入去噪，交互预测。本节对前三个模块进行消融实验，验证各模块的有效性。本节把 DNN 作为基础深度推荐模型，在 MovieLens-1M 数据集上展开消融实验。PMEC(generator)代表只使用生成器构建模块；PMEC(optimize)代表使用生成器构建模块和嵌入空间优化模块；PMEC(enhancing)代表使用生成器构建模块和嵌入去噪模块。PMEC 代表使用所有模块。AUC 和 GAUC 指标的实验结果如图 3-7 所示，做了十次实验，阴影部分代表波动，实线代表十次实验的平均值。



(a) 消融实验的 AUC 指标



(b) 消融实验的 GAUC 指标

3.3. 效果提升了多少?

这个随便编，例如：最终离线GAUC：0.85→0.86，提高一个百分点。

(或者直接看代码里的结果也行)

4. 训练配置说明

python3.6.3, tensorflow1.12.0。

只有这两个是强制要求的，剩下的包缺啥装啥即可，如果遇到报错，一般是版本不对，把报错信息扔到百度，看看需要装什么版本即可。

为什么用tensorflow1.12.0?

答：因为公司用的都是tensorflow1.12.0，pytorch只适合科研，但tensorflow1.12.0计算速度更快。所以最好提前熟悉一下，进了公司以后轻松一点。

5. 数据说明

csv文件全都处理成了tfrecord文件，这样的话对于tensorflow1来说，计算更快。

5.1. 第一种类型的数据

以这三个文件为例【train_onehot_a_w_ngb_hist、train_onehot_b_w_ngb_hist、train_onehot_c_w_ngb_hist】

用于训练MAML，共197列

- 占1列的6个特征，共计6列：user_id, movie_id, gender, age, occ, release year
- 占5列的2个特征，共计10列：title (mul-hot), genres (mul-hot)。如果把文本转化为5列的特征呢？建立词典（这个方法网上一大堆资料，比如，“满江红”，满是1，江是2，红是3，“西红柿首富”，西是4.....编好以后，有些movie的title可能会很长，只取它的前5个字，然后取对应编号，就形成了一个5位的特征。如果movie的title不足5个字，就补0。例如上面的例子，满江红就是1, 2, 3, 0, 0。还有其他实现方式。）

以上16列，可以标志着一个user点击了一个item。然后要把相似的10个老item的数据拼上来，共计160列。

- 占20列的特征，共计20列：最近点击过当前movie的20个user的id。
- 最后一列是label，，共计1列。

所以全部加起来，是197列。

5.2. 第二种类型的数据

用于训练大推荐模型，共计17列

- 占1列的6个特征，共计6列：user_id, movie_id, gender, age, occ, release year
- 占5列的2个特征，共计10列：title (mul-hot), genres (mul-hot)。如果把文本转化为5列的特征呢？建立词典（这个方法网上一大堆资料，比如，“满江红”，满是1，江是2，红是3，“西红柿首富”，西是4.....编好以后，有些movie的title可能会很长，只取它的前5个字，然后取对应编号，就形成了一个5位的特征。如果movie的title不足5个字，就补0。例如上面的例子，满江红就是1, 2, 3, 0, 0。还有其他实现方式。）

- 最后一列是label, , 共计1列。

所以全部加起来, 是17列。

6. 代码说明

代码实现从简 (实现简单, 但不妨碍面试讲的牛逼)。

- 优化点1: 真实实践中, 用老item的embedding作为信息增量, 发现效果一般, 就删了。但这个优化思想是正确的, 真实的工业界很多人这么做。所以面试的时候可以讲。
- 优化点2: 经过一番尝试, 只计算正样本上的loss, 效果会好。但是面试还是按照文档里讲。
- 优化点3: 正常讲。

7. 简历怎么写

背景: 新item产生时, 训练数据不足, 导致其id embedding得不到很好的训练, 进而导致推荐效果差。我基于meta learning缓解了冷启动问题。

1. 构造了 id embedding 生成器, 为冷启动的 item 生成良好的初始 id embedding。并使用 Meta learning 中MAML进行训练。训练时的优化目标考虑了两方面: 完全冷启动阶段, 应该最小化预测损失; 预热阶段, 即数据不断增多时, 应该最小化预热损失。
2. 基于对比学习构造了数据增强任务, 优化了生成器, 使得生成的id embedding同源相近, 异源相远。可以使得id embedding 的特征空间更加均匀, 提高了生成器所生成的id embedding的质量。
3. 冷启动item数据少, 其id embedding容易受到噪声的干扰, 例如用户的错误点击。使用了冷启动item有限的交互序列进行了去噪, 降低了离群点的影响。

做完所有优化后, 离线GAUC: 0.85→0.86, 提高一个百分点, 并做了消融实验, 各个部分都是有效的。

8. 常见面试问题

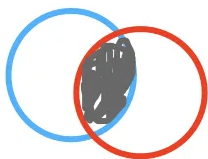
问题一：优化目标是什么？

答：优化目标其实就是问你损失函数的设计。

- 在完全冷启动阶段，期望给冷启动物品一个不错的初始嵌入，最小化冷启动的预测损失（0数据的时候）。
- 在预热阶段，冷启动物品有了少量的交互数据，期望最小化预热阶段的预测损失。
- 对比损失（这个说不说都行，主要是前两个loss，一定要给面试官点出来）。

问题二：相比于传统的梯度下降（以batch为单位更新梯度），maml的优势在哪呢？

答：MAML希望通过学习不同的任务，学到他们的共性，来快速适应新任务。



（就是这个图）

从初始化的参数到最优参数，如果对于一个正常的item来说，是没有问题的。但是我们研究的是冷启动问题，它没有足够多的数据，去让参数训到最优状态。所以，用很少的数据训练出来的参数一定是过拟合的。所以maml的一种思路是：先给模型一点点数据（也就是maml的训练集，也叫支撑集），模型一看，这么简单，直接更新跑到了过拟合的位置，然后我们让它涨涨教训，给他一个测试集（也就是maml查询集）测一下，傻眼了，loss很高，模型会反思，奥，我刚刚跑到过拟合的地方去了，没把参数更新好。所以，把这两阶段的loss都更新到刚开始的初始参数上，相当于pua模型，以后别给我跑到过拟合的地方去。经过很多任务的学习，模型再也不敢跑到过拟合的地方去了。

问题三：遇到最大的困难是什么？

在进行老代新的时候，如何判断两个item比较相似。首先用id embedding做余弦相似度肯定是不OK的，因为新item的id embedding几乎和随机初始化没有任何区别。所以选择了用属性相似来判断item是否相似。然后再讲怎么判断相似。后续优化，也想过不同属性的重要性可能是不同的，但尝试给不同属性加权（比如主观上认为特征1可能比特征2重要一些，就设计成：如果特征1相同就加一分，特征2相同就加两分，但试了试发现没啥效果，就还是对所有特征一视同仁）

问题四：在几个数据集上进行了实验？

在电影数据集、阿里广告数据集、CIKM的比赛数据集上都进行了实验，效果都是正向的。

问题五：优化3的序列长度是多少？

30，不足的补0。

问题六：做数据增强时，子集的划分方式？为什么这么划分？

首先在冷启动物品的特征集中随机选择一个种子特征f_seed，然后利用余弦相似度计算和种子特征最相关的一半特征，然后把种子特征和这一半特征作为一个子集，其余特征作为另一个子集。这样做可以保证两个子集的信息差异最大化，更有利于基于对比学习的数据增强任务。

冷启动物品的属性特征集要先划分为两个子集，才能执行上述的数据增强任务。不同的划分方式，会导致不同的结果。不同的特征之间存在相关性，如果划分后的两个子集很多特征都高度相关，那它们通过生成器生成的id嵌入也很容易相似，这就失去了对比学习的意义。为了解决这个问题，首先在冷启动物品的特征集中随机选择一个种子特征f_seed，然后利用余弦相似度计算和种子特征最相关的一半特征，然后把种子特征和这一半特征作为一个子集，其余特征作为另一个子集。这样做可以保证两个子集的信息差异最大化，更有利于基于对比学习的数据增强任务。

其他学生的问題：

1.怎么证明ID嵌入生成器得到的ID嵌入比随机生成的ID嵌入要好呢？

答：先用随机生成的id embedding，和其他特征拼接后经过DNN后会得到一个预估结果，计算一次GAUC1；再用冷启动框架生成一个id embedding，和其他特征拼接后经过DNN，也能计算出一个GAUC2。GAUC1<GAUC2，证明了冷启动框架的有效性。

2.有没有对比过直接把和当前物品最相似的10个物品的ID嵌入当成新物品的ID初始嵌入的效果呢，不需要生成器？

答：因为咱们的项目是基于MAML的冷启动项目，是高级的用法。而面试官提的这种方法，是低级的替代方法，你就直接说，试过，但是没这个效果好，就OK了。事实上，这两种方法哪种效果更好，都有可能，但是如果你面试讲简单版本的，那还有啥讲的意义呢？

3.对比学习中的负采样怎么做？

先把这个batch内不同的item拿出来，然后用这些不同的item的embedding做负样本。（这个在代码层面也很好实现，大模型就能直接教你怎么写，就不细讲了）我当时实现的比较简单的原因是，我发现用最标准的流程，效果还不如我这样简单实现一下，我们老师又是一个只看结果的人，所以我就改成了简单版本。

4.这里old2是作为冷启动的训练集吗？如果是的话，那优化点1的找相似k的老item是从old1里匹配吗？

是的。